

ACTIVIDAD 2
PRUEBAS DEL SISTEMA INTELIGENTE DE BUSQUEDA DE
RUTAS



Estudiante:

Rodrigo Bonilla Torres

Profesor(a):

Sandra Bautista

Corporación Universitaria Iberoamericana - IBERO

Ingeniería de Software – Inteligencia Artificial

Bogotá D.C, Colombia

2026

Documento de evidencias de pruebas funcionales y automáticas realizadas sobre el sistema desarrollado en Python.

1. Objetivo de las pruebas

Verificar el correcto funcionamiento del sistema inteligente encargado de determinar una ruta de menor costo entre dos estaciones de un modelo simplificado de transporte masivo. Las pruebas validan la base de conocimiento, las reglas lógicas, el cálculo heurístico y el algoritmo de búsqueda A*.

2. Descripción del sistema

El sistema representa las estaciones y sus conexiones mediante un grafo. Cada conexión posee un costo de desplazamiento y cada estación tiene coordenadas relativas utilizadas para calcular una heurística mediante distancia euclidiana. A* evalúa cada alternativa utilizando la expresión $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado y $h(n)$ es la estimación del costo restante hasta el destino.

Nota: los nombres de algunas estaciones toman como referencia el sistema de transporte de Bogotá, pero las conexiones, coordenadas y costos corresponden a un modelo académico simplificado y no representan rutas, tiempos ni distancias operacionales reales.

3. Resumen de pruebas realizadas

Prueba	Resultado esperado	Resultado
Portal Sur → Portal Norte	Ruta válida, costo 37	Exitosa
Portal Suba → Portal Américas	Ruta válida, costo 39	Exitosa
Pruebas automáticas	7 pruebas aprobadas	7/7 - OK

4. Caso de prueba 1: Portal Sur → Portal Norte

Se seleccionó Portal Sur como origen y Portal Norte como destino. El algoritmo A* encontró una ruta de 11 estaciones y 10 tramos, con un costo total estimado de 37 unidades. El resultado confirma que el sistema puede recorrer el grafo en sentido sur-norte y acumular correctamente los costos de cada conexión.

```
=====
MEJOR RUTA ENCONTRADA
=====
1. Portal Sur
2. NQS Calle 30 Sur
3. Tercer Milenio
4. Avenida Jimenez
5. Calle 26
6. Calle 45
7. Calle 63
8. Calle 72
9. Calle 100
10. Calle 127
11. Portal Norte

Número de estaciones: 11
Número de tramos: 10
Costo total estimado: 37
```

Figura 1. Resultado de la búsqueda Portal Sur → Portal Norte.

5. Caso de prueba 2: Portal Suba → Portal Américas

Se seleccionó Portal Suba como origen y Portal Américas como destino. El sistema construyó dinámicamente una ruta de 10 estaciones y 9 tramos, con costo total estimado de 39 unidades. Este caso es especialmente útil porque obliga al algoritmo a desplazarse entre diferentes ramales del grafo y a evaluar alternativas mediante la función heurística.

```
=====
MEJOR RUTA ENCONTRADA
=====
1. Portal Suba
2. Suba Calle 100
3. Calle 100
4. Calle 72
5. Calle 63
6. Calle 45
7. Calle 26
8. Avenida Jimenez
9. Ricaurte
10. Portal Americas

Número de estaciones: 10
Número de tramos: 9
Costo total estimado: 39
```

Figura 2. Resultado de la búsqueda Portal Suba → Portal Américas.

6. Pruebas automáticas

Se implementaron siete pruebas automáticas utilizando el módulo **unittest** de la biblioteca estándar de Python. Las pruebas verifican estaciones existentes e inexistentes, la regla de origen y destino iguales, conexiones directas, el valor de la heurística en el destino y los costos esperados de dos rutas completas.

```
rhobt@Rodri MINGW64 /c/git/SistemaInteligente (main)
$ py -m unittest discover -s tests -v
test_conexion_directa (test_sistema.TestSistemaInteligente.test_conexion_directa)
Verifica una relación almacenada ... ok
test_estacion_existente (test_sistema.TestSistemaInteligente.test_estacion_existente)
Verifica que una estación registrada ... ok
test_estacion_inexistente (test_sistema.TestSistemaInteligente.test_estacion_inexistente)
Verifica que una estación desconocida ... ok
test_heuristica_destino (test_sistema.TestSistemaInteligente.test_heuristica_destino)
La heurística de una estación respecto ... ok
test_mismo_origen_destino (test_sistema.TestSistemaInteligente.test_mismo_origen_destino)
Comprueba la regla que determina si ... ok
test_ruta_portal_sur_portal_norte (test_sistema.TestSistemaInteligente.test_ruta_portal_sur_portal_norte)
Verifica la búsqueda A* entre ... ok
test_ruta_suba_americas (test_sistema.TestSistemaInteligente.test_ruta_suba_americas)
Verifica la ruta utilizada en nuestro ... ok

-----
Ran 7 tests in 0.013s

OK
```

Figura 3. Ejecución de las siete pruebas automáticas con resultado OK.

Prueba automática	Aspecto validado
test_estacion_existente	Base de conocimiento
test_estacion_inexistente	Validación de datos
test_mismo_origen_destino	Regla lógica
test_conexion_directa	Relaciones del grafo
test_heuristica_destino	$h(\text{destino}) = 0$
test_ruta_portal_sur_portal_norte	Búsqueda A* y costo 37
test_ruta_suba_americas	Búsqueda A* y costo 39

7. Análisis de resultados

Los resultados obtenidos muestran que el sistema integra correctamente los tres elementos principales planteados para la actividad: representación del conocimiento mediante un grafo, reglas lógicas para validar y consultar el conocimiento, y búsqueda heurística mediante A*. En los casos funcionales, la ruta no está codificada de forma directa; se construye durante la ejecución a partir de las conexiones disponibles.

La heurística utiliza la distancia euclidiana entre coordenadas relativas. Durante la búsqueda, A* prioriza los nodos de acuerdo con $f(n) = g(n) + h(n)$, combinando el costo ya recorrido con una estimación de lo que falta para llegar al objetivo.

8. Conclusiones

Las pruebas funcionales produjeron los costos esperados (37 y 39 unidades), mientras que las siete pruebas automáticas finalizaron satisfactoriamente con resultado **OK**. Con base en estas evidencias, el sistema cumple los comportamientos definidos para el modelo académico y permite demostrar de forma práctica el uso de reglas, representación del conocimiento y técnicas de búsqueda heurística en Python.